

CONCEPT NOTE

Conflux

Six proven open source engines, covering every stage from raw ingestion to live analytics — orchestration, storage, table format, catalog, transformation, and query — fully wired into one connected platform instead of six separate projects.

Written after personally hand-wiring these six tools together, stage by stage, and personally feeling every seam where one stage didn't know the next one existed.

NOTE TO REVIEWER

1. Core architecture: Conflux is a thin control-plane application sitting in front of six independent, already-proven open source engines — Airflow, RustFS, Iceberg, Polaris, dbt, and Trino — auto-generating the connection config between them at startup instead of replacing or reimplementing any of their actual functionality.
2. Product framing: it's presented the way Moheet AI presents Chat, Agent, Automate, Operations, Devices, Insights, and Knowledge — one brand, several clearly named components, each doing one real job, operated from a single console rather than experienced as separate, unrelated tools.
3. Day-to-day operating model: on first run, Conflux provisions storage, bootstraps the catalog, wires the query engine's connector, and registers the orchestrator's pipelines automatically, then exposes one login and one health view for all six going forward.
4. Authorship: the wording throughout this document was drafted with AI assistance, but every technical claim, error message, and fix described in it was personally built, run, and verified against a real working implementation before being written down — none of it is speculative copy.
5. Ownership: if this concept is approved, I will take full personal responsibility for completing the project end to end.

WHAT'S INSIDE

Every Stage, One Connected Platform

Conflux doesn't replace any of these six projects. It's the missing glue layer that makes them behave like one product, in lifecycle order — orchestrate, store, format, catalog, transform, query — instead of six separate projects a person has to connect by hand.



1 · ORCHESTRATE

Apache Airflow

Schedules and runs every pipeline. Conflux auto-generates each DAG and pre-wires its dbt runner config, instead of a hand-written Python file and two easy-to-miss invocation-mode settings.



2 · STORE

RustFS

Holds the physical Parquet and metadata files. Conflux provisions its bucket and access keys automatically and hands them to the catalog and query engine without anyone typing them twice.



3 · FORMAT

Apache Iceberg

Defines how a table is actually laid out as files — schema, snapshots, atomic commits. Conflux never touches this layer directly; it just makes sure every other layer agrees on where to find it.



4 · CATALOG

Apache Polaris

Tracks which snapshot is current for every table. Conflux auto-bootstraps its catalog and namespaces on first start — no manual curl script, no hand-written bootstrap credentials.



5 · TRANSFORM

dbt

Compiles and runs every SQL model. Conflux ships the fix for dbt's default table materialization conflicting with a catalog that blocks purge-on-drop, built in — no custom macro required, no silent 403 on the second run.



6 · QUERY

Trino

Runs the actual SQL a person or dashboard asks for. Conflux auto-generates its catalog connector config from the running catalog and storage services instead of a hand-edited properties file.

Where this idea came from

Building the lakehouse lab that this document grew out of meant standing up Airflow, RustFS, Iceberg, Polaris, dbt, and Trino by hand — six genuinely excellent, genuinely open source projects, each covering one real stage of getting data from a raw source into a query a person can actually run, and each one built with absolutely no awareness that the other five exist. Getting them to actually work together meant writing a bootstrap script that logs into Polaris, obtains a token, and creates a catalog with a hand-typed storage location. It meant hand-writing a Trino connector properties file with the catalog's REST URL, OAuth credentials, and S3 endpoint typed in manually, in the right format, with no validation until Trino either started or didn't. It meant discovering — the hard way, mid-project — that Polaris keeps its catalog state in memory and forgets everything the moment its container restarts, with no warning and no built-in recovery. It meant a separate, equally manual fight on the orchestration side: Airflow's own dbt runner needed an isolated Python environment baked into a custom Docker image just to avoid a dependency conflict, and its config for rendering versus running that dbt project turned out to be two separate settings that both silently default to the wrong mode unless both are set by hand. And on the transform side specifically, dbt's own default behavior for rebuilding a table — rename the old one, build the new one, drop the backup — quietly failed against a catalog that refuses to let anything purge on drop, a 403 that only showed up on the second run of a model that had worked fine on the first.

None of that is a flaw in any of the six projects individually. Each one does its own job well. The problem is that "well" stops at each project's own boundary — nothing in the open Iceberg ecosystem is responsible for the seams between them, and every team adopting this stack ends up writing the same glue code, hitting the same bootstrap problems, and rediscovering the same gotchas independently, stage by stage.

Six excellent engines, covering six real stages, and zero of them know the other five exist.

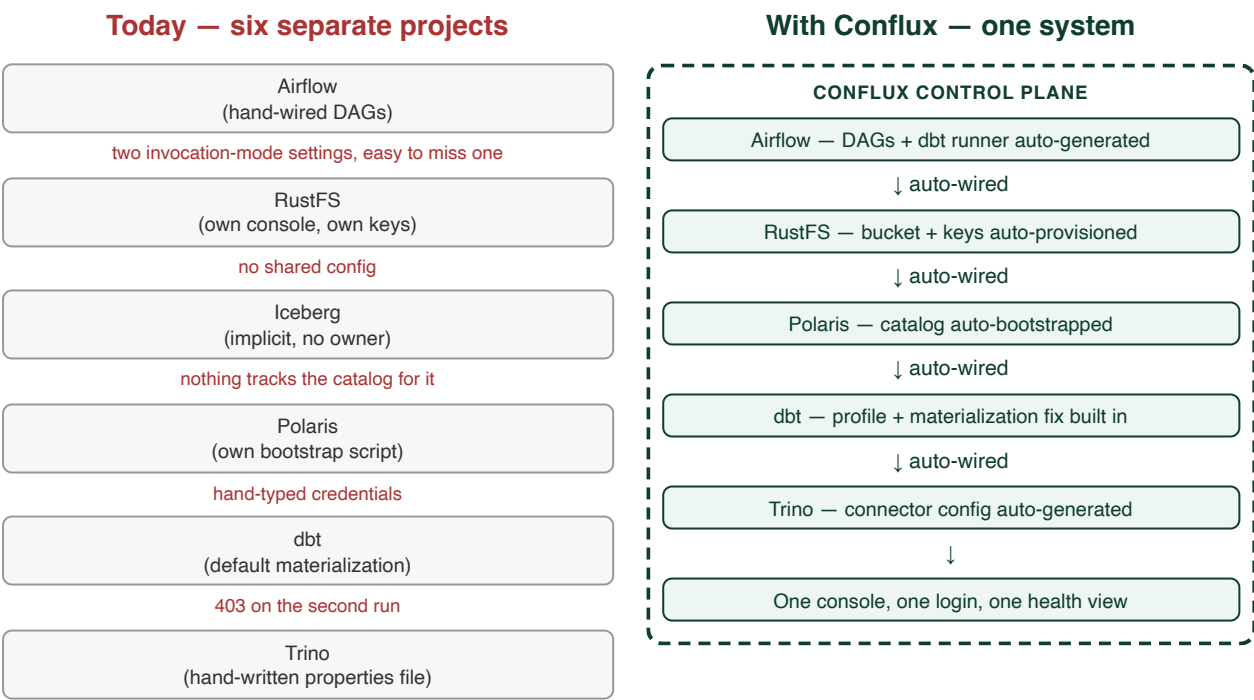
The idea

Conflux is a single application whose only job is that missing glue layer — but the reframe worth stating plainly is that it now covers a complete lifecycle, not just a pile of infrastructure. Orchestration decides when something runs. Storage holds the bytes. A table format decides how those bytes represent a table. A catalog decides which version of that table is current. Transformation decides what the data actually becomes. Compute answers

the query. Install Conflux once, and all six stages stand up as one connected system — every connection between them generated automatically from the others' actual running state, not typed in by hand and hoped to be correct.

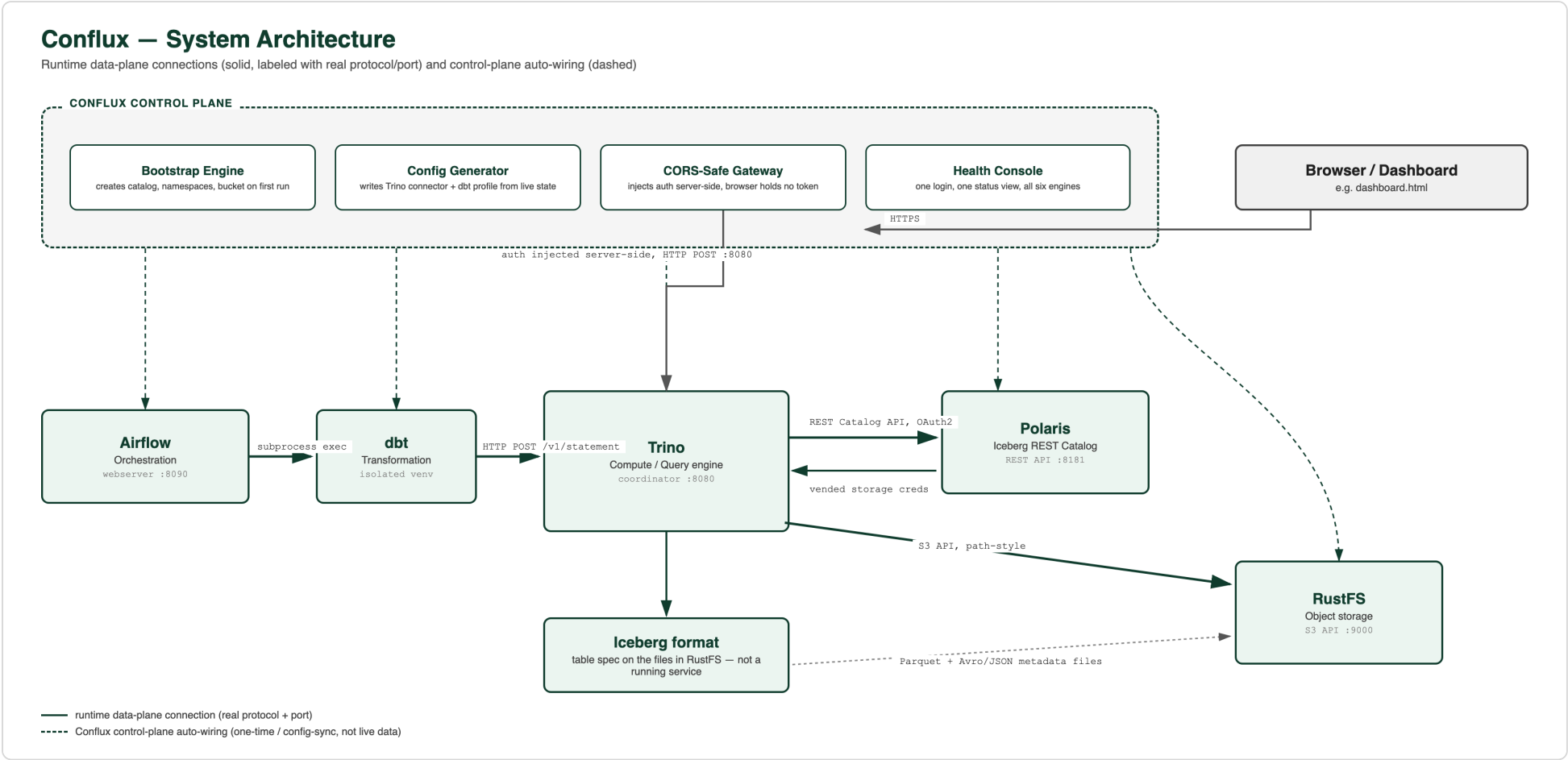
This is the same shape of idea as tools that bundle several independent open source services into one cohesive product with one login and one dashboard — Postgres plus an auth service plus a storage service plus a realtime layer, presented and operated as a single thing even though several separate open source projects sit underneath. Conflux applies that same idea to the full open lakehouse lifecycle: Airflow, RustFS, Iceberg, Polaris, dbt, and Trino become one thing to install, one thing to look at, one thing to operate — start to finish, not just the storage and query pieces in the middle.

Before Conflux, and after



System architecture

Solid arrows are real runtime connections, labeled with the actual protocol and port each one uses. Dashed arrows are Conflux's own one-time or config-sync auto-wiring — not live data traffic.



What Conflux actually does

Capability	What it replaces
One-command install	Separately reading each project's own setup docs and reconciling six different configuration formats by hand.
Auto-bootstrapped catalog	A hand-written curl script that logs into Polaris, exchanges credentials for a token, and creates the first catalog — the exact script this lab needed to build from scratch.
Auto-generated Trino connector	Manually typing a properties file with the catalog's REST URL, OAuth secret, S3 endpoint, and region, with no feedback until the query engine either starts or silently fails.
One unified web console	Five separate surfaces — Airflow's own UI, Trino's query UI, Polaris's REST API (no UI at all), RustFS's object browser, dbt's own CLI output — none of which know the others exist.
One login, one credential set	Separate access keys for RustFS, separate OAuth client id/secret for Polaris, no shared identity between them.
Restart-safe catalog state	Polaris's in-memory catalog forgetting every table registration on restart, silently, with the physical Parquet files still sitting untouched in storage and no automatic way to reconnect them.
Auto-generated DAGs	Hand-writing a Python DAG file per pipeline, plus keeping Cosmos's separate render-time and run-time invocation-mode settings in sync by hand — miss one and the DAG fails to parse or fails to run, with two different error messages depending on which one was missed.
Built-in, CORS-safe API layer	Hand-building a reverse proxy just to let a browser-based app query the engine directly — a real problem this lab hit and had to solve from scratch.
Swappable storage backend	RustFS by default for a fully free local setup, one config value away from pointing at real S3, GCS, or Azure Blob without touching the catalog or compute layer.

Why this is worth building

Every one of the problems above was hit directly, not imagined. The bootstrap script, the hand-written connector file, the in-memory catalog surprise, the missing CORS support — these came from actually operating this exact six-tool stack, not from guessing what might go wrong with it. That's the honest case for Conflux: it isn't solving a hypothetical integration problem, it's automating a specific, repeatable set of manual steps that anyone adopting this open source combination will hit in more or less the same order.

Who this is for

- Teams who want the openness and portability of Iceberg without personally becoming the integration layer between six separate projects.
- Anyone evaluating an open lakehouse stack before committing to a managed platform, who wants the real components running as one coherent system, not six browser tabs.
- Small teams and solo builders who don't have the bandwidth to maintain hand-written bootstrap scripts and connector files as each underlying project's own config format changes across versions.

Where this could go next

- A guided first-run flow that provisions storage, catalog, and compute in one pass and reports exactly what got created, instead of leaving that implicit.
- A watchdog that detects a Polaris restart and automatically re-registers existing tables from the physical files still present in storage, closing the exact gotcha this lab had to document and work around manually.
- Optional support for swapping Trino or Polaris for compatible alternatives, since Conflux's actual value is the glue layer, not a hard lock to any one of the six engines specifically.

Written from direct, hands-on experience wiring these six specific open source projects together — every problem described here was actually hit, not hypothesized.